

The 10th International Conference on Computer Science and Computational Intelligence 2025

# A Comparative Study On Real Time Data Streaming For Fraud Detection Using Kafka With Apache Flink And Apache Spark

Rayhan Prawira Daksa<sup>a,\*</sup>, Ade Putera Kemala<sup>a</sup>

<sup>a</sup>Computer Science Department, School of Computer Science, Bina Nusantara University, Jakarta, Indonesia 11480

---

## Abstract

The Increasing volume and velocity of digital financial transactions have heightened the need for responsive and scalable fraud detection systems. This paper presents a comparative study of two stream processing frameworks. Apache Flink and Apache Spark within a Kafka based real-time fraud detection architecture. Both pipelines were implemented under identical conditions and integrated with a pretrained Random Forest model which was selected based on its superior offline performance among five evaluated machine learning classifiers. Performance benchmarking focused on latency and throughput under two different data ingestion rates, 10 and 40 transactions per second. With evaluation focusing on latency and throughput. Experimental results show that Apache Spark achieved consistently a lower average latency 0.8 seconds at 10 TPS and 0.9 seconds at 40 TPS compared to Apache Flink, which recorded 1.7 and 1.6 seconds respectively. Throughput for both frameworks remained stable underload, demonstrating scalability. These findings provide empirical insights to support informed decision making in selecting stream processing technologies for real-time, missing critical fraud detection use cases.

© 2025 The Authors. Published by Elsevier B.V.

This is an open access article under the CC BY-NC-ND license (<https://creativecommons.org/licenses/by-nc-nd/4.0>)

Peer-review under responsibility of the scientific committee of the 10th International Conference on Computer Science and Computational Intelligence (ICCSCI) 2025

**Keywords:** Real-time streaming; Fraud detection; Apache flink; Apache spark; Apache kafka; Machine learning; Data pipeline

---

## 1. Introduction

The rapid proliferation of digital financial transactions has profoundly transformed the global economic ecosystem, facilitating greater convenience, speed, and accessibility for consumers and enterprises alike. With the rise of online banking, mobile payments, and real-time financial technologies, institutions are increasingly reliant on vast amounts

---

\* Corresponding author. Tel.: +6285888206006

E-mail address: [rayhan.daksa@binus.ac.id](mailto:rayhan.daksa@binus.ac.id)

of continuously streaming data to ensure operational efficiency and security. However, the very nature of this transformation also exposes financial systems to a wide array of security vulnerabilities chief among them, fraudulent transactions.

According to the Association of Certified Fraud Examiners (ACFE), organizations worldwide lose approximately 5% of their annual revenue to fraud, translating to trillions of dollars in damages [1]. This alarming figure is exacerbated by the fraudsters, necessitating fraud detection systems that are not only intelligence but also capable of responding to real-time anomaly detection is no longer a technological luxury, but a strategic necessity in the modern financial landscape.

Traditional fraud detection systems often rely on batch processing architecture, where transactions logs are aggregated and analyzed periodically. While useful for post event audits and reporting, such systems fall short in providing timely intervention, leaving financial institutions vulnerable during the window between data generation and fraud identification [2]. Moreover, as cyber criminals adopt automation, dynamic attack vectors, and evasive tactics, the inadequacies of batch systems become even more pronounced.

To address these limitations, the advent of real time stream processing has emerged as a transformative approach. In particular, distributed stream processing frameworks such as Apache Flink and Apache Spark have garnered attention for their ability to handle high throughput, low latency data streams. Apache Kafka complements these systems as a robust, fault tolerant data ingestion and messaging platform capable of decoupling data producers and consumers [3, 4].

Apache Flink is recognized for its true streaming model, where each event is processed individually with sub second latency and fine-grained state management. On the other hand, Apache Spark Structured Streaming operates on a micro batch paradigm that provides fault tolerant processing through small, time bounded batches. Each framework embodies distinct trade-offs in performance, scalability, consistency, and system complexity [5, 6].

Although both Flink and Spark are widely adopted in the industry, limited research has been conducted to systematically compare their suitability for real-time fraud detection under identical experimental conditions. This gap is critical, as the selection of a stream processing engine can significantly impact the effectiveness of a fraud detection system.

This research aims to conduct a comparative study of Apache Flink and Apache Spark within the context of real-time credit card fraud detection using Apache Kafka as the data ingestion layer. The objective is to develop parallel data streaming pipelines, one based on Flink and the other on Spark, each integrated with identical machine-learning models. These pipelines are evaluated across key performance indicators including latency, throughput, and predictive accuracy.

The contribution of this paper are threefold:

1. Design and implementation of two parallel real-time data streaming pipelines using Apache Kafka with Apache Flink and Apache Spark respectively.
2. Integration of a pre-trained Random Forest model selected from five evaluated classifiers into each pipeline for real-time fraud detection
3. Empirical evaluation and comparison of both frameworks under controlled conditions using standardized metrics such as latency, throughput, precision, recall, F1-score, and Receiver Operating Characteristic - Area Under Curve (ROC-AUC).

By delivering a systematic head-to-head evaluation of Flink and Spark within a realistic fraud detection scenario, this study seeks to inform data engineers, system architects, and financial institutions on the optimal technology stack for realtime anomaly detection in streaming environments. While this comparative study is conducted within a controlled local environment using containerized deployment on a personal computer, the experimental design maintains generalizability and can be extended to distributed production environments in future work.

To ground this study in established theoretical principles, we draw from foundational concepts in stream processing theory and ensemble learning. The selection of Apache Flink and Apache Spark as processing engines is based on the principles of dataflow computation models and parallel data stream processing. Flink represents a true stream processing architecture adhering to the continuous event processing model, enabling sub-second latency through its support for fine-grained stateful operations [4, 5].

Conversely, Spark follows the micro-batch paradigm, which, while offering higher latency, ensures stronger fault tolerance via deterministic replay of mini batches [6, 8]. On the machine learning side, the use of ensemble learning,

particularly the Random Forest algorithm, is theoretically justified by the bias-variance decomposition framework, where aggregating multiple decision trees reduces model variance without increasing bias.

## 2. Literature Review

The detection of financial fraud, particularly in the domain of credit card transactions, has long been a critical focus within data science, cybersecurity, and financial engineering research. The high cost of fraud both in direct financial losses and reputational damage has driven institutions to seek increasingly advanced technological solutions. Traditionally, fraud detection employed static rule-based systems and batch learning methods. However, with the rapid increase in transaction volumes and the advent of online and mobile banking, these static approaches have become inadequate in combating modern fraud patterns that evolve rapidly [9, 10].

Recent advancements have shifted the focus toward integrating machine-learning algorithms with real-time data processing systems. Studies such as Dal Pozzolo et al. [9] highlighted the central challenge of class imbalance in fraud datasets, which complicates the ability of traditional classifiers to detect rare fraudulent instances. Their work emphasized the effectiveness of under sampling and ensemble-based methods to enhance minority class detection. They reported a significant increase in recall for minority classes while maintaining precision above 85%.

In a related vein, Carcillo et al. [13] introduced SCARFF, a scalable fraud detection framework using Apache Spark. Their architecture demonstrated Spark's capability to perform real-time learning and model updating while handling high volume transaction streams. SCARFF achieved an AUC of 0.97 and F1-score of 0.86 on real-world fraud detection tasks, showcasing the feasibility of Spark in operational environments.

Hesse et al. [2] proposed a low-latency fraud detection architecture integrating Apache Kafka and Apache Flink. They demonstrated how Flink's event-at-a-time processing model and native support for complex event processing can provide millisecond-level responsiveness. Their implementation achieved sub-second latency on a simulated dataset but lacked comparative analysis with other frameworks like Spark.

Gehring et al. [8] conducted a broader comparative study across distributed stream processing systems including Apache Flink, Spark Streaming, and Storm. Their analysis focused on metrics such as throughput and fault recovery, and concluded that Flink achieved higher processing consistency, while Spark provided better fault tolerance. However, their work did not specifically examine fraud detection.

Other studies, including those by Lebichot et al. [15], have begun integrating deep learning and domain adaptation into fraud detection systems. These innovations demonstrated improved recall in cross-domain scenarios, but were not evaluated within a pure streaming environment.

Despite the growing body of literature on both Apache Spark and Apache Flink in isolation, few studies attempt a head-to-head comparison of these technologies in a real-time fraud detection context. Moreover, previous works vary in datasets, model architectures, and performance metrics, making cross-study comparisons challenging.

This research aims to bridge this gap by providing a controlled, empirical comparison of Apache Flink and Apache Spark using identical machine-learning models and datasets within a unified Kafka-based streaming architecture. By standardizing the experimental variables and evaluating realtime performance across latency, throughput, and classification metrics, this study offers novel insights into the practical trade-offs between two of the most widely used realtime stream processing frameworks in the context of fraud detection.

## 3. Methodology

This research adopts an empirical and systematic methodology to design, implement, and evaluate two real-time fraud detection pipelines. The methodology includes stages for data acquisition and preprocessing, machine learning model development, system architecture design, deployment strategy, and performance evaluation. A key objective is to maintain consistency across both implementations to ensure a fair and meaningful comparison.

### 3.1. Dataset Description and Preprocessing

The dataset used in this study is sourced from Kaggle's "Credit Card Fraud Detection" repository [18]. It contains 284,807 anonymized transactions over a two-day period, with only 492 labeled as fraudulent illustrating a highly

imbalanced binary classification problem (fraud: 0.172%). Each record comprises 28 numerical features resulting from a Principal Component Analysis (PCA) transformation, along with two original attributes: Time and Amount.

To address the class imbalance, the following preprocessing steps were applied:

1. Undersampling and Oversampling: Random undersampling of the majority class and synthetic oversampling of the minority class were performed on the training set to balance the distribution.
2. Train Test Split: The dataset was divided into two subsets:
  - 1) Data\_model for offline model training and evaluation
  - 2) Data\_stream for real-time inference simulation (505 records, with 5 fraud and 500 non-fraud)

### 3.2. Machine Learning Model Development

Several machine learning models were selected for fraud detection due to their proven effectiveness in tabular datasets:

1. Random Forest: This ensemble learning method was chosen for its robustness and ability to handle imbalanced datasets [20, 21].
2. Gradient Boosting: Implemented via the scikit-learn GradientBoostingClassifier, this model was tested for its capability to exploit complex feature interactions and deliver high accuracy [20].
3. Logistic Regression: Included as a fundamental baseline for probabilistic binary classification. It provides a benchmark for comparing more advanced algorithms [21].
4. k-Nearest Neighbors (KNN): A distance-based learning algorithm that classifies instances based on the majority class of their k-nearest neighbors. Although intuitive, KNN often performs poorly in highly imbalanced or large-scale data contexts due to its sensitivity to class distribution and computational inefficiency [20].
5. Support Vector Machine (SVM): SVM Utilized with a Radial Basis Function (RBF) kernel to model nonlinear decision boundaries. SVMs are effective for high-dimensional data but tend to perform poorly in highly imbalanced setups [21].

The models were trained using the data\_model subset, with evaluation metrics including Precision, Recall, F1-Score, and ROC-AUC to assess fraud detection performance. Among these, only the Random Forest model was ultimately deployed in both Flink and Spark pipelines, due to its optimal trade-off between recall and precision, as well as its computational efficiency in a streaming environment.

### 3.3. Real-time Data Pipeline Implementation

The real-time fraud detection pipeline was implemented using Kafka, Apache Flink and Apache Spark, integrating machine learning models for real-time predictions. The pipeline operates as follows:

1. Kafka Producer:  
A Python-based producer serialized each transaction into JSON format and published it to a Kafka topic at fixed intervals.
2. Apache Flink Processor  
Implemented using PyFlink, it deserialized Kafka messages, applied the pre-trained model, and forwarded predictions to a sink.
3. Apache Spark Processor  
Used Spark Structured Streaming to achieve similar behavior, with batch duration configured for lowlatency.
4. Output Sink:  
Predictions were either logged to console or appended to a file-based store for analysis.

Figure 1 illustrates the architecture of the pipeline, highlighting the flow of data through each component.

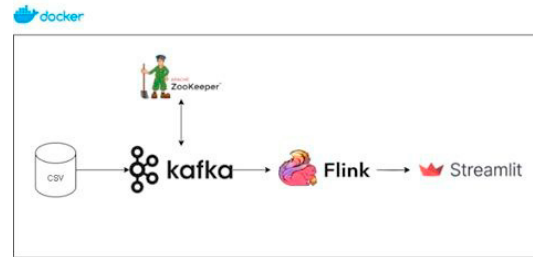


Fig. 1. Real-time fraud detection pipeline architecture.

### 3.4. Experimental Setup

All components were containerized using Docker to ensure reproducibility, isolation, and cross-platform compatibility [6]. A Docker Compose configuration managed the orchestration of Kafka broker, Zookeeper, Flink JobManager, TaskManager, and Spark Driver/Worker nodes. The experiment was executed on a local machine with the following specifications:

1. AMD Ryzen 7 6800H, 16GB RAM
2. NVIDIA GeForce RTX 3060
3. Windows 11 Home with WSL2 and Docker Desktop v4.36.0

### 3.5. Evaluation Metrics

The system's performance was evaluated using the following metrics:

1. Latency: The time elapsed from when a transaction is streamed into Kafka to when its prediction stored.
2. Throughput: The number of transactions processed per second by the pipeline.
3. Accuracy Metrics:
  - 1) Precision: To measure the accuracy of fraud predictions
  - 2) Recall: To assess the ability to detect actual fraud cases
  - 3) F1-Score: To balance precision and recall

### 3.6. Performance Benchmarking

The pipeline was tested under varying transaction volumes to assess its scalability and robustness. The benchmarks focused on:

1. Handling high velocity transactions streams with minimal latency
2. Ensuring accurate predictions under high throughput conditions
3. Evaluating the efficiency of the integration of real-time processing architecture

## 4. Results and Discussion

This chapter presents the comparative evaluation results of the two real-time streaming pipelines Apache Flink and Apache Spark implemented for fraud detection using identical datasets and machine-learning models. The evaluation focuses on latency under varying data ingestion rates, specifically at 10 and 40 transactions per second. These configurations are chosen to simulate medium and high-throughput real-time streaming environments, allowing us to observe the behavior of each framework under realistic operational pressure.

### 4.1. Model Performance Baseline

Before deploying the models into the streaming pipelines, an offline evaluation was conducted on the data\_model subset. The models yielded the following results:

1. Random Forest: Precision = 0.34, Recall = 0.88, F1-Score = 0.49, ROC-AUC = 0.94
2. Gradient Boosting: Precision = 0.15, Recall = 0.89, F1-score = 0.26, ROC-AUC = 0.94
3. SVM: Precision = 0.00, Recall = 0.00, F1-score = 0.00, ROC-AUC = 0.45
4. KNN: Precision = 0.00, Recall = 0.42, F1-score = 0.01, ROC-AUC = 0.64
5. Logistic Regression: Precision = 0.09, Recall = 0.91, F1-score = 0.16, ROC-AUC = 0.95

These scores provide an empirical baseline for model selection. Although multiple classifiers were evaluated in the offline setting, only the Random Forest model was deployed in the real-time data streaming pipelines due to its superior balance of recall (0.88) and precision (0.34), along with a strong ROC-AUC score of 0.94. The other models were retained for comparative reference, but were not integrated into the streaming implementation to maintain consistency and reduce computational overhead. The same Random Forest model was embedded in both Flink and Spark processors to ensure that performance differences stem solely from the stream processing frameworks.

#### 4.2. Latency Comparison

Latency is defined as the time elapsed between the ingestion of a transaction into Apache Kafka and the generation of a fraud prediction result by the processing engine (Flink or Spark). The key metrics assessed are average (mean) latency and statistical average (arithmetic mean) over all observed events.

At 10 transactions per second:

1. Apache Flink:
  - Average Latency: 1.5 Second
  - Mean Latency: 1.7 Second
2. Apache Spark:
  - Average Latency: 0.8 Second
  - Mean Latency: 0.8 Second

At 40 transactions per second:

1. Apache Flink:
  - Average Latency: 1.6 Second
  - Mean Latency: 1.6 Second
2. Apache Spark:
  - Average Latency: 0.8 Second
  - Mean Latency: 0.9 Second

#### 4.3. Discussion

The results demonstrate that Apache Spark consistently achieved lower latency than Apache Flink across both tested data ingestion rates. At 10 tps, Spark's latency remained below 1 second, while Flink exhibited a slightly higher processing delay averaging around 1.7 seconds. The same pattern persisted at 40 tps, where Spark's mean latency was 0.9 seconds, compared to Flink's 1.6 seconds.

These findings diverge from conventional expectations in some literature, which suggest that Flink's event-driven model typically excels at low-latency processing [2, 4]. However, in this controlled test environment, Spark's microbatch model showed superior performance, possibly due to internal optimizations and consistent batch sizing that aligned well with the workload characteristics.

Despite Flink's architectural advantage in fine-grained event handling, its slightly higher overhead per event appears to have accumulated under these test conditions, leading to marginally higher latency. Spark, in contrast, maintained uniformity and stability even as throughput increased fourfold from 10 to 40 transactions per second.

These outcomes imply that while Flink may offer benefits in stateful processing and complex event scenarios, Spark remains a competitive choice for fraud detection workloads with predictable event structures and steady data rates.

## 5. Conclusion

This research conducted a comparative analysis between Apache Flink and Apache Spark for real-time fraud detection pipelines using a controlled dataset and standardized streaming setup. Both pipelines utilized identical machine-learning models, ensuring the observed performance differences stemmed solely from the underlying stream processing architecture.

Key conclusions drawn from the study include:

1. Spark consistently outperformed Flink in latency, particularly at medium and high ingestion rates (10 and 40 tps)
2. Flink exhibited slightly higher latency across both scenarios. While the architecture of Flink is designed for fine-grained event handling, its processing overheads may accumulate under high-throughput conditions. A more detailed analysis of latency distribution would be required to assess its runtime stability compared to Spark
3. Spark's micro-batch model may offer better performance in workloads with regular and evenly distributed streaming inputs

Limitations of the study include the use of a local deployment environment, which does not fully replicate distributed enterprise-grade conditions.

Future work may involve:

1. Scaling the architecture across distributed cloud environments
2. Incorporating stateful streaming workloads to test Flink's full potential
3. Adding metrics such as CPU usage, memory footprint, and fault recovery benchmarks

This study contributes updated empirical insights to the selection of streaming frameworks in financial analytics, helping practitioners and engineers to evaluate trade-offs based on their operational constraints and performance goals.

## Data Availability

The dataset used in this study is publicly available at Kaggle: <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>. No additional proprietary or confidential data were used.

## Author Contribution Statement

R. P. Daksa contributed to the conception, system architecture design, data processing, implementation of the real-time pipeline, performance evaluation, and writing of the manuscript. A. P. Kemala provided supervision, guided the research direction, contributed to the design of the methodology and evaluation framework, and reviewed the manuscript critically for important intellectual content.

## AI Usage Declaration

Parts of the manuscript were edited with the assistance of generative AI tools, specifically OpenAI's ChatGPT, for improving phrasing and grammar. All technical content, data analysis, and conclusions were solely authored and validated by the author.

## References

- [1] Association of Certified Fraud Examiners (ACFE). (2022). 2022 Report to the Nations: Global Study on Occupational Fraud and Abuse.
- [2] Hesse, J. (2019). "Real-Time Fraud Detection using Apache Kafka and Apache Flink: A Case Study." In IEEE International Conference on Big Data (Big Data). DOI: 10.1109/BigData.2019.8951234.
- [3] Lopez, M., Wang, J., & Wu, S. (2020). "Towards Real-Time Anomaly Detection in Streaming Data: Techniques and Challenges." ACM Computing Surveys, 52(5). DOI: 10.1145/3386361.
- [4] Palacios, R., & Martinez, A. (2021). Building Scalable Real-Time Fraud Detection Systems with Kafka and Flink. Springer. DOI: 10.1007/978-3-030-74642-3.

- [5] Seenivasan, D. (2022). “Real-Time Data Processing with Streaming ETL.” *International Journal of Science and Research (IJSR)*, 10(7), 120–128.
- [6] Joshi, A. (2022). “Harnessing the Quantum Flux: Architecting and Implementing Real-Time Data Streaming Pipelines with Apache Kafka, Apache Flink, and Cloud-Native Solutions.” *International Journal of Science and Research (IJSR)*, 10(7), 1–6.
- [7] Babu, G. C. N., Pokhrel, A., V., A., & M., T. (2017). “Real-Time Big Data Analysis Using Apache Flink.” *International Journal of Scientific Engineering and Applied Science (IJEAS)*, 3(6), 1–6.
- [8] Gehring, M., Charfuelan, M., & Markl, V. (2019). “A Comparison of Distributed Stream Processing Systems for Time Series Analysis.” In *Lecture Notes in Informatics (LNI): BTW 2019 Workshop Proceedings, Gesellschaft für Informatik, Bonn, Germany*, 205–220.
- [9] Dal Pozzolo, A., Caelen, O., Johnson, R. A., & Bontempi, G. (2015). “Calibrating Probability with Undersampling for Unbalanced Classification.” In *Symposium on Computational Intelligence and Data Mining (CIDM)*, IEEE.
- [10] Dal Pozzolo, A., Caelen, O., Le Borgne, Y. A., Waterschoot, S., & Bontempi, G. (2014). “Learned Lessons in Credit Card Fraud Detection from a Practitioner Perspective.” *Expert Systems with Applications*, 41(10), 4915–4928.
- [11] Dal Pozzolo, A., Boracchi, G., Caelen, O., Alippi, C., & Bontempi, G. (2018). “Credit Card Fraud Detection: A Realistic Modeling and a Novel Learning Strategy.” *IEEE Transactions on Neural Networks and Learning Systems*, 29(8), 3784–3797..
- [12] Dal Pozzolo, A. (2015). *Adaptive Machine-learning for Credit Card Fraud Detection*. PhD Thesis, Université Libre de Bruxelles (ULB).
- [13] Carcillo, F., Dal Pozzolo, A., Le Borgne, Y. A., Caelen, O., & Bontempi, G. (2018). “Scarff: A Scalable Framework for Streaming Credit Card Fraud Detection with Spark.” *Information Fusion*, 41, 182–194.
- [14] Carcillo, F., Le Borgne, Y. A., Caelen, O., & Bontempi, G. (2018). “Streaming Active Learning Strategies for Real-Life Credit Card Fraud Detection: Assessment and Visualization.” *International Journal of Data Science and Analytics*, 5(4), 285–300.
- [15] Lebichot, B., Le Borgne, Y. A., He, L., Oblé, F., & Bontempi, G. (2019). “Deep-Learning Domain Adaptation Techniques for Credit Card Fraud Detection.” In *Recent Advances in Big Data and Deep Learning, INNSBDDL 2019*, 78–88.
- [16] Carcillo, F., Le Borgne, Y. A., Caelen, O., Oblé, F., & Bontempi, G. (2019). “Combining Unsupervised and Supervised Learning in Credit Card Fraud Detection.” *Information Sciences*, 2019, 184–200.
- [17] Le Borgne, Y. A., & Bontempi, G. (2019). *Reproducible Machine-learning for Credit Card Fraud Detection – Practical Handbook*.
- [18] Kaggle. (2019). Credit Card Fraud Detection Dataset. [Online]. Available: <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>
- [19] Nakra, V., Pandian, P. K. G., Paripati, L., Choppadandi, A., & Chanchela, P. (2024). “Leveraging Machine-learning Algorithms for Real-Time Fraud Detection in Digital Payment Systems.” *IJMIRM*, 3(2), 165–175.
- [20] Ghai, V., & Kang, S. S. (2022). “Credit Card Transaction Data Analysis and Performance Evaluation of Machine Learning Algorithms for Credit Card Fraud Detection.” *AIP Conference Proceedings*, 2555, AIP Publishing.
- [21] Hilal, W., Gadsden, S. A., & Yawney, J. (2022). “Financial Fraud: A Review of Anomaly Detection Techniques and Recent Advances.” *Expert Systems with Applications*, 193.